Back to calendar

16% progress

Mandatory deliverable

LAB | Use Community Modules Lab

Skills you will learn

Infrastructure as Code: Modules | Development Workflow: Code Reusability

Lab M4.06 - Use Community Modules from Terraform Registry

Repository:
https://github.com/cloud-engineering-bootcamp/ce-lab-terraform-registry-modules

Activity Type: Individual **Estimated Time:** 30-45 minutes **Submission:** GitHub Repository

Learning Objectives

- [ ] Browse and find modules in Terraform Registry
- [ ] Understand module documentation
- [ ] Use official AWS modules from registry
- [ ] Configure module inputs properly
- [ ] Combine multiple registry modules
- [ ] Compare custom vs community modules

Prerequisites

- [ ] Completed Lab M4.05 (Create Module)
- [ ] Understanding of module concepts
- [ ] Basic VPC and EC2 knowledge

Introduction

Don't reinvent the wheel! Terraform Registry hosts thousands of pre-built, tested modules. Learn to leverage community modules to build infrastructure faster and with best practices baked in.

Scenario

Instead of creating everything from scratch, use battle-tested community modules:

- **VPC Module**: Official AWS VPC module (10M+ downloads)
- **EC2 Module**: Pre-configured EC2 instances
- **Security Group Module**: Common security group patterns

Your Task

What you'll do:

- Browse Terraform Registry
- Deploy VPC using official module
- Deploy EC2 instances using registry module
- Create security groups with module
- Compare with your custom module

Time limit: 30-45 minutes

Step-by-Step Instructions

Step 1: Explore Terraform Registry

Visit**https://registry.terraform.io/**

Search for "aws vpc"

Click on `terraform-aws-modules/vpc/aws`

Review: Module description
Usage examples
Input variables
Outputs
Download count
Version history

Key observations:

- 10M+ downloads
- Well-documented
- Many configuration options
- Active maintenance

Step 2: Create Lab Project

```
mkdir -p ~/ce-labs/m4-06-registry-modules
cd ~/ce-labs/m4-06-registry-modules

cat > main.tf <<'EOF'
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}
EOF
```

Copy Explain

Step 3: Use Official VPC Module

Add to `main.tf`:

```
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "5.1.0"  # Always specify version!

  name = "registry-vpc"
  cidr = "10.0.0.0/16"

  azs             = ["us-east-1a", "us-east-1b", "us-east-1c"]
  private_subnets = ["10.0.1.0/24", "10.0.2.0/24", "10.0.3.0/24"]
  public_subnets  = ["10.0.101.0/24", "10.0.102.0/24", "10.0.103.0/24"]

  enable_nat_gateway = true
  single_nat_gateway = true  # Cost savings
  enable_dns_hostnames = true

  # Tags
  tags = {
```

Copy Explain

```
  Environment = "dev"
  Project     = "registry-demo"
  ManagedBy   = "Terraform"
 }

 vpc_tags = {
  Name = "registry-vpc"
 }
}
```

Deploy:

```
terraform init
terraform plan
terraform apply -auto-approve
```

Copy Explain

Expected outcome: Complete VPC with subnets, NAT Gateway, route tables in ~3 minutes.

Step 4: Add EC2 Instance Module

Add to `main.tf`:

```
# Data source for latest Amazon Linux 2 AMI
data "aws_ami" "amazon_linux_2" {
 most_recent = true
 owners      = ["amazon"]

 filter {
  name   = "name"
  values = ["amzn2-ami-hvm-*-x86_64-gp2"]
 }
}

# EC2 Instance Module
module "ec2_instance" {
 source  = "terraform-aws-modules/ec2-instance/aws"
 version = "5.5.0"

 name = "registry-web-server"

 instance_type          = "t2.micro"
 ami                    = data.aws_ami.amazon_linux_2.id
 vpc_security_group_ids =
[module.security_group_web.security_group_id]
 subnet_id              = module.vpc.public_subnets[0]

 # User data
 user_data = <<-EOF
             #!/bin/bash
```

Copy Explain

```
            yum update -y
            yum install -y httpd
            systemctl start httpd
            systemctl enable httpd
            echo "<h1>Hello from Registry Module!</h1>" >
/var/www/html/index.html
            EOF

  tags = {
    Environment = "dev"
    ManagedBy   = "Terraform"
  }
}
```

Step 5: Add Security Group Module

Add to `main.tf` :

Copy Explain

```
module "security_group_web" {
  source  = "terraform-aws-modules/security-group/aws"
  version = "5.1.0"

  name        = "web-server-sg"
  description = "Security group for web server"
  vpc_id      = module.vpc.vpc_id

  # Ingress rules
  ingress_cidr_blocks = ["0.0.0.0/0"]
  ingress_rules       = ["http-80-tcp", "https-443-tcp"]

  # Custom SSH rule
  ingress_with_cidr_blocks = [
    {
      from_port   = 22
      to_port     = 22
      protocol    = "tcp"
      description = "SSH from anywhere"
      cidr_blocks = "0.0.0.0/0"
    }
  ]

  # Egress
  egress_rules = ["all-all"]

  tags = {
    Environment = "dev"
  }
}
```

Step 6: Add Outputs

Create `outputs.tf`:

Copy Explain

```
output "vpc_id" {
  description = "VPC ID from registry module"
  value       = module.vpc.vpc_id
}

output "public_subnets" {
  description = "Public subnet IDs"
  value       = module.vpc.public_subnets
}

output "private_subnets" {
  description = "Private subnet IDs"
  value       = module.vpc.private_subnets
}

output "instance_id" {
  description = "EC2 instance ID"
  value       = module.ec2_instance.id
}

output "instance_public_ip" {
  description = "Public IP of EC2 instance"
  value       = module.ec2_instance.public_ip
}

output "web_url" {
  description = "URL to access web server"
  value       = "http://${module.ec2_instance.public_ip}"
}

output "security_group_id" {
  description = "Security group ID"
  value       = module.security_group_web.security_group_id
}
```

Step 7: Deploy Complete Infrastructure

Copy Explain

```
terraform init -upgrade  # Download new modules
terraform plan
terraform apply -auto-approve
```

Expected outcome:

```
Apply complete! Resources: 25+ added

Outputs:

instance_public_ip = "54.123.45.67"
vpc_id = "vpc-abc123"
web_url = "http://54.123.45.67"
```

Step 8: Test the Deployment

Copy Explain

```
# Get web URL
WEB_URL=$(terraform output -raw web_url)

# Test web server
curl $WEB_URL

# Expected: <h1>Hello from Registry Module!</h1>

# Or open in browser
echo "Open in browser: $WEB_URL"
```

Step 9: Compare Module Approaches

Create comparison document `COMPARISON.md` :

Copy Explain

```
# Custom Module vs Registry Module Comparison

## Custom Module (Lab M4.05)
**Pros:**
- Full control over implementation
- Customized to exact needs
- Learning experience
- No external dependencies

**Cons:**
- Time-consuming to develop
- Need to maintain and update
- May miss best practices
- Need to write tests

## Registry Module
**Pros:**
- Battle-tested (10M+ downloads)
- Best practices built-in
- Active maintenance
- Comprehensive documentation
```

- Quick to implement

Cons:
- Less customization
- Need to understand module internals
- Version management required
- May include unused features

Recommendation
- **Use Registry**: For common patterns (VPC, EC2, RDS)
- **Use Custom**: For company-specific requirements
- **Combine Both**: Registry for base, custom for specialized needs

Step 10: Explore More Modules

Try other popular modules:

Copy Explain

```
# S3 Bucket Module
module "s3_bucket" {
  source  = "terraform-aws-modules/s3-bucket/aws"
  version = "3.15.0"

  bucket = "my-registry-test-bucket"
  acl    = "private"

  versioning = {
    enabled = true
  }

  server_side_encryption_configuration = {
    rule = {
      apply_server_side_encryption_by_default = {
        sse_algorithm = "AES256"
      }
    }
  }
}

# RDS Module
module "db" {
  source  = "terraform-aws-modules/rds/aws"
  version = "6.3.0"

  identifier = "registry-db"

  engine               = "mysql"
  engine_version       = "8.0"
  family               = "mysql8.0"
  major_engine_version = "8.0"
  instance_class       = "db.t3.micro"
```

```
  allocated_storage = 20

  db_name  = "demodb"
  username = "admin"
  password = "CHANGE_ME_IN_PRODUCTION"  # Use Secrets
Manager!

  subnet_ids             = module.vpc.private_subnets
  vpc_security_group_ids =
[module.security_group_db.security_group_id]

  tags = {
    Environment = "dev"
  }
}
```

Documentation & Submission

Create README.md

Copy Explain

```
# Lab M4.06 - Terraform Registry Modules

## Overview
Deployed complete infrastructure using official Terraform Registry
modules.

## Modules Used
- **VPC Module**: `terraform-aws-modules/vpc/aws` v5.1.0
- **EC2 Module**: `terraform-aws-modules/ec2-instance/aws` v5.5.0
- **Security Group**: `terraform-aws-modules/security-group/aws`
v5.1.0

## Infrastructure Created
- VPC with 3 public and 3 private subnets
- NAT Gateway for private subnet internet access
- EC2 instance running Apache web server
- Security group allowing HTTP/HTTPS/SSH

## Benefits of Registry Modules
- Pre-tested and maintained
- Best practices included
- Fast deployment
- Comprehensive documentation

## Comparison
See COMPARISON.md for custom vs registry module analysis.

## Access
```

Web server accessible at: [terraform output web_url]

Create Repository

```
git init
git add .
git commit -m "Lab M4.06: Use Terraform Registry Modules

- Deployed VPC using official AWS VPC module
- Created EC2 instance with registry module
- Configured security groups with module
- Compared custom vs registry approaches"

gh repo create ce-lab-terraform-registry-modules --public
git push -u origin main
```

Copy Explain

Troubleshooting

Issue 1: Version Constraint Error

Error:

```
Error: Failed to query available provider packages
```

Solution: Specify exact version:

```
version = "5.1.0"  # Not "~> 5.0"
```

Copy Explain

Issue 2: Module Not Found

Error:

```
Error: Module not installed
```

Solution:

```
terraform init -upgrade
```

Copy Explain

Grading Rubric (100 points)

Criteria	Points
VPC module deployed	25
EC2 module deployed	25
Security group module used	20
All resources working	15
Comparison document	10
Documentation	5

Key Takeaways

✅ **Terraform Registry** - Thousands of pre-built modules ✅ **Official modules** - Well-tested, widely used ✅ **Always version** - Specify module versions ✅ **Read docs** - Understand inputs and outputs ✅ **Combine approaches** - Registry + custom modules ✅ **Best practices** - Learn from community modules

Next Lab: M4.07 - Terraform AWS Networking (VPC Deep Dive)

LAB | Use Community Modules

Lab

URL

Enter the URL here

Submit lab

Previous

Week 4 - Infrastructure as Code with Terraform: Day 3 - Terraform Modules

Terraform Modules Quiz

Next